DSE-3: Data Science – (Unit-3)

What is data cleaning?

Data cleaning is the process of fixing or removing incorrect, corrupted, incorrectly formatted, duplicate, or incomplete data within a dataset. When combining multiple data sources, there are many opportunities for data to be duplicated or mislabeled. If data is incorrect, outcomes and algorithms are unreliable, even though they may look correct. There is no one absolute way to prescribe the exact steps in the data cleaning process because the processes will vary from dataset to dataset. But it is crucial to establish a template for our data cleaning process.

Difference between data cleaning and data transformation:

Data cleaning is the process that removes data that does not belong in our dataset. Data transformation is the process of converting data from one format or structure into another. Transformation processes can also be referred to as data wrangling, or data munging, transforming and mapping data from one "raw" data form into another format for warehousing and analyzing.

Steps involved in Data Cleaning:

1. **Removal of unwanted observations**: This includes deleting duplicate/ redundant or irrelevant values from your dataset. Duplicate observations most frequently arise during data collection and irrelevant observations are those that don't actually fit the specific problem that we're trying to solve.
    - Redundant observations alter the efficiency by a great extent as the data repeats and may add towards the correct side or towards the incorrect side, thereby producing unfaithful results.
    - Irrelevant observations are any type of data that is of no use to us and can be removed directly.
2. **Fixing Structural errors**: The errors that arise during measurement, transfer of data, or other similar situations are called structural errors. Structural errors include typographical error, the same attribute with a different name, unlabeled classes, i.e. separate classes that should really be the same, or inconsistent capitalization. For example, we may find "N/A" and "Not Applicable" both appear, but they should be analyzed as the same category.
3. **Filtering Unwanted outliers**: Outliers are those **data points that are really far from the rest of the data points**. They may arise due to errors in data entry or measurement, or just because there's variation in the population we're looking at. Outliers can cause problems with certain types of models. For example, linear regression models are less robust to outliers than decision tree models. Generally, we should not remove outliers until we have a legitimate reason to remove them. Sometimes, removing them improves performance, sometimes not. So, one must have a good reason to remove the outlier, such as suspicious measurements that are unlikely to be part of real data.

4. **Handling missing data**: We can't ignore missing data because many algorithms will not accept missing values. There are a couple of ways to deal with missing data. Neither is optimal, but both can be considered.
   i. As a first option, we can drop observations that have missing values, but doing this will drop or lose information, so we must be mindful of this before we remove it.
   ii. As a second option, we can input missing values based on other observations; again, there is an opportunity to lose integrity of the data because we might be doing so from assumptions and not from the actual observations.

Some data cleansing tools

- Openrefine
- Trifacta Wrangler
- TIBCO Clarity
- Cloudingo
- IBM Infosphere Quality Stage

Obtaining data from the web: There is a lot of web data that is useful to social scientists, including Social media, News media, Government publications, Organizational records etc. There are two ways to get data from the web.

1. **Webscraping** - i.e., user-facing websites for humans.
2. **Web APIs** - i.e., application-facing for computers.

Web scraping: Web scraping is an automatic method to obtain large amounts of data from websites. Most of this data is unstructured data in an HTML format which is then converted into structured data in a spreadsheet or a database so that it can be used in various applications. There are many different ways to perform web scraping to obtain data from websites. These include **using online services, particular API's or even creating our code for web scraping from scratch.** Many large websites, like Google, Twitter, Facebook, StackOverflow, etc. have API's that allow us to access their data in a structured format. This is the best option, but there are other sites that don't allow users to access large amounts of data in a structured form or they are simply not that technologically advanced. In that situation, it's best to use Web Scraping to scrape the website for data.

Web scraping requires two parts, namely the **crawler** and the **scraper**. The crawler is an artificial intelligence algorithm that browses the web to search for the particular data required by following the links across the internet. The scraper, on the other hand, is a specific tool created to extract data from the website. The design of the scraper can vary greatly according to the complexity and scope of the project so that it can quickly and accurately extract the data.

How does Web Scrapers Work? Web Scrapers can extract all the data on particular sites or the specific data that a user wants. Ideally, it is best if we specify the data we want so that the web scraper only extracts that data quickly. For example, we want to scrape an Amazon page for the types of juicers available, but

we might only want the data about the models of different juicers and not the customer reviews.
So, when a web scraper needs to scrape a site, first the URLs are provided. Then it loads all the HTML code for those sites and a more advanced scraper might even extract all the CSS and Javascript elements as well. Then the scraper obtains the required data from this HTML code and outputs this data in the format specified by the user. Mostly, this is in the form of an Excel spreadsheet or a CSV file, but the data can also be saved in other formats, such as a JSON file.

Different Types of Web Scrapers: Web Scrapers can be divided on the basis of many different criteria, including Self-built or Pre-built Web Scrapers, Browser extension or Software Web Scrapers, and Cloud or Local Web Scrapers.

We can have **Self-built Web Scrapers** but that requires advanced knowledge of programming. And if we want more features in our Web Scrapper, then we need even more knowledge. On the other hand, pre-built **Web Scrapers** are previously created scrapers that we can download and run easily. These also have more advanced options that we can customize.

Browser extensions Web Scrapers are extensions that can be added to our browser. These are easy to run as they are integrated with our browser, but at the same time, they are also limited. Any advanced features that are outside the scope of our browser are impossible to run on Browser extension Web Scrapers.

But **Software Web Scrapers** don't have these limitations as they can be downloaded and installed on our computer. These are more complex than Browser web scrapers, but they also have advanced features that are not limited by the scope of our browser.

Cloud Web Scrapers run on the cloud, which is an off-site server mostly provided by the company that we buy the scraper from. These allow our computer to focus on other tasks as the computer resources are not required to scrape data from websites.

Local Web Scrapers, on the other hand, run on our computer using local resources. So, if the Web scrapers require more CPU or RAM, then our computer will become slow and not be able to perform other tasks.

Web APIs: API stands for **Application Programming Interface**. Broadly defined, an API is a set of rules and procedures that facilitate interactions between computers and their applications.

A very common type of API is the **Web API**, which (among other things) allows users to query a remote database over the internet.

Web APIs take on a variety of formats, but the vast majority adheres to a particular style known as **Representational State Transfer** or **REST**. What makes these "RESTful" APIs so convenient is that we can use them to query databases using URLs. RESTful Web APIs are all around us…

Consider a simple Google search: Ever wonder what all that extra stuff in the address bar was all about? In this case, the full address is Google's way of

sending a query to its databases requesting information related to the search term “golden state warriors.”

In fact, it looks like Google makes its query by taking the search terms, separating each of them with a “+”, and appending them to the link “https://www.google.com/search?q= Therefore, we should be able to actually change our Google search by adding some terms to the URL and following the general format…

Learning how to use RESTful APIs is all about learning how to format these URLs so that we can get the response we want.

Using APIs in R: There are two ways to collect data through APIs in R:

1. **Plug-n-play packages:** Many common APIs are available through user-written R Packages. These packages offer functions that “wrap” API queries and format the response. These packages are usually much more convenient than writing our own query, so it is worth searching around for a package that works with the API we need.
2. **Writing our own API request:** If no wrapper function is available, we have to write our own API request and format the response ourselves using R. This is trickier, but definitely achievable.

Collecting Twitter Data with RTweet: Twitter actually has two separate APIs and there are several packages of R for accessing and searching Twitter.

1. The **REST API** allows us to read and write Twitter data. For research purposes, this allows us to search the recent history of tweets and look up specific users.
2. The **Streaming API** allows us to access public data flowing through Twitter in real time. It requires our R session to be running continuously, but allows us to capture a much larger sample of tweets while avoiding rate limits for the REST API.